Experiment- 5

Transfer Learning with Deep CNNs

1. Aim

To study transfer learning in deep CNNs by fine-tuning pretrained models such as VGG19 and MobileNetV2 on the Oxford Flowers dataset. To analyze how layer freezing, fine-tuning depth, learning rate, and dataset size affect feature extraction, training dynamics, representation quality, gradient flow, overfitting, and final inference performance. To implement and execute the models step by step to understand their architecture, transfer learning workflow, training process, and inference behavior.

2. Theory

Introduction to Transfer Learning

In most traditional supervised machine learning approaches, it is assumed that the training data and test data are represented using the same feature space and are drawn from the same probability distribution. Under this assumption, models can perform well only when sufficient training data is available. However, in many real-world scenarios, collecting high-quality data is expensive and time-consuming, which becomes a major bottleneck in practical applications.

Semi-supervised learning attempts to reduce data requirements by using unlabelled data along with training data, but it still assumes that training and test data belong to the same domain and distribution. Active learning reduces the manual effort by selecting the most informative samples, but it is limited by the available resources and may still not provide enough data to train an accurate classifier.

Domain and Task Definitions (Formal Notation)

Domain (D)

A domain consists of:

- A feature space X
- A marginal distribution $P(x)$

So, a domain is:

$$D = \{X, P(x)\}$$

Task (T)

A task consists of:

- A label space Y

- A predictive function f, which maps the feature space X to the label space Y, i.e.,

$$f: X \rightarrow Y$$

where Xrepresents the feature space and Yrepresents the label space.

So, a task is defined as:

$$T = \{Y, f(\cdot)\}$$

Transfer Learning Definition

Given:

- Source domain $D_S$and task T_S
- Target domain $D_T$and task T_T

Transfer learning aims to improve the target predictive function $f_T(\cdot)$in $D_T$using the knowledge from $D_S$and T_S, where:

$$D_S \neq D_T \text{or} T_S \neq T_T$$

This means:

- The domains may have different feature spaces or distributions
- The tasks may have different label spaces or predictive distributions

When both domains and tasks are the same, the learning becomes a traditional machine learning problem.

Homogeneous vs Heterogeneous Transfer Learning

Homogeneous Transfer Learning

Homogeneous Transfer Learning refers to a type of transfer learning in which the source domain and target domain share the same or highly similar feature representation. In this case, the input data from both domains lies in an overlapping feature space, meaning that the source and target datasets are represented in a comparable manner. This type of transfer learning is widely applied in practical scenarios where the domains involve similar data, such as transferring knowledge from one image dataset to another (image → image).

Heterogeneous Transfer Learning

Heterogeneous Transfer Learning is a more complex form of transfer learning in which the feature representations of the source and target domains differ. Here, the datasets are described using

different feature spaces, making direct transfer difficult. Since the representations do not overlap, it is necessary to apply feature-mapping or transformation techniques to establish a relationship between the source and target domains. Due to feature-space mismatches, heterogeneous transfer learning is considered more challenging than homogeneous transfer learning and requires additional processing steps to effectively align or bridge the domains.

Deep CNNs

Convolutional neural networks or CNNs, are a specialized kind of neural network for processing data that has a known, grid-like topology. Examples include time-series data, which can be thought of as a 1D grid taking samples at regular time intervals, and image data, which can be thought of as a 2D grid of pixels. Convolutional networks have been tremendously successful in practical applications. The name "convolutional neural network" indicates that the network employs a mathematical operation called convolution. Convolution is a specialized kind of linear operation. Convolutional networks are simply neural networks that use convolution in place of general matrix multiplication in at least one of their layers.

Deep Convolutional Neural Networks (CNNs), also called ConvNets, have become one of the most important architectures for large-scale image recognition. Their success in tasks such as image classification and object recognition was made possible by large public image repositories like ImageNet and high-performance computing resources such as GPUs and distributed computing systems. A major milestone in the progress of deep CNNs was the ImageNet Large-Scale Visual Recognition Challenge (ILSVRC), which provided a benchmark platform for evaluating and comparing image classification systems. Early approaches were based on shallow feature representations, but later deep CNN-based models achieved significantly better performance and became dominant in visual recognition tasks.

In deep learning, transfer learning is typically implemented using a pretrained CNN backbone (trained on large datasets such as ImageNet). The pretrained model learns rich hierarchical representations:

- Early layers learn general patterns (edges, textures)
- Middle layers learn shapes and structures
- Deep layers learn high-level category-specific features

For a small dataset like Oxford Flowers, transfer learning improves performance by:

- Reducing training time
- Preventing overfitting
- Reusing learned feature extraction ability

Hence, fine-tuning pretrained models such as VGG19 or MobileNetV2 is an effective strategy for building accurate classification models with limited training samples.

Merits of Transfer learning with deep CNNs

- **Reduced Training Time:**
  Transfer learning reduces the training time because the model does not need to be trained from scratch.
- **Works Well on Small Datasets:**
  It provides good accuracy even when the target dataset is small by using pretrained knowledge.
- **Better Feature Extraction:**
  Pretrained CNNs already learn powerful features such as edges, textures, and shapes, thereby improving performance.
- **Reduced Overfitting:**
  Since the model starts with pretrained weights, the chances of overfitting are reduced.
- **Faster Convergence:**
  The model converges faster because the weights are already optimized on large datasets such as ImageNet.

Demerits of Transfer Learning with Deep CNNs

- **Domain Dependency:**
  If the source and target datasets are very different, transfer learning may not give good results.
- **Complexity in Layer Freezing:**
  Choosing which layers to freeze or unfreeze needs careful tuning and experimentation.
- **Risk of Overfitting during Fine-tuning:**
  Fine-tuning on a very small dataset can lead to overfitting if not properly controlled.
- **High Computation for Large Models:**
  Large models like VGG19 require high memory and more computation, making training slow.
- **Hardware Requirement:**
  Deep CNN training may still require GPU support for faster training and performance.

3. Code and Result

Step 1: Import Libraries

INPUT:

```
# Importing Libraries

import os

import numpy as np

import matplotlib.pyplot as plt
```

```
import tensorflow as tf
import seaborn as sns
import json
from sklearn.metrics import classification_report
from tensorflow.keras.applications.vgg19 import preprocess_input
from tensorflow.keras.layers import GlobalAveragePooling2D, Dense, Dropout
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.applications import VGG19
print("All the required libraries are imported.\n")
```

OUTPUT:

All the required libraries are imported successfully.

Step 2: Define Parameters

INPUT:

```
# Defining Paths & Parameters
base_path = '/kaggle/input/oxford-102-flower-dataset/102 flower/flowers'
json_path = '/kaggle/input/oxford-102-flower-dataset/102 flower/cat_to_name.json'
img_size = 224
batch_size = 32
epochs = 150
patience = 15
learning_rate = 1e-5
output_dir = './vgg19_outputs'
os.makedirs(output_dir, exist_ok=True)
```

```
print(f"Parameters for training are set as:")
print(f"  Image Size: {img_size}")
print(f"  Batch Size: {batch_size}")
print(f"  Epochs: {epochs}")
print(f"  Patience: {patience}")
print(f"  Learning Rate: {learning_rate}\n")
```

OUTPUT:

```
Parameters for training are set as:
  Image Size: 224
  Batch Size: 32
  Epochs: 150
  Patience: 15
  Learning Rate: 1e-05
```

Step 3: Visualize Dataset

INPUT:

```
# Visualzing Dataset
with open(json_path, 'r') as f:
    real_names = json.load(f)
numeric_classes = sorted(os.listdir(os.path.join(base_path, 'train')))
num_classes = len(numeric_classes)
class_names = [real_names[c] for c in numeric_classes]
print("\n================ CLASS MAPPING TABLE ================")
print(f"Number of classes: {num_classes}\n")
print(f"{'Folder':<10} | {'Flower Name'}")
print("-" * 50)
```

```
for f, cname in zip(numeric_classes, class_names):
    print(f"{f:<10} | {cname}")
print("=====================================================\n")
train_raw = tf.keras.utils.image_dataset_from_directory(
    os.path.join(base_path, "train"),
    image_size=(img_size, img_size),
    batch_size=9,          # small batch for visualization
    label_mode='int',
    shuffle=True,
    verbose=0)
# Showing samples from dataset
def show_samples(ds_raw, numeric_class_list, name_map, n=9):
    plt.figure(figsize=(8, 8))
    for batch_x, batch_y in ds_raw.take(1):
        imgs = batch_x.numpy().astype("uint8")
        labels = batch_y.numpy()
        break
    rows = int(np.ceil(np.sqrt(n)))
    for i in range(n):
        ax = plt.subplot(rows, rows, i+1)
        plt.imshow(imgs[i])
        folder_id = numeric_class_list[int(labels[i])]
        cls_name = name_map[folder_id]
        ax.set_title(cls_name, fontsize=9)
        ax.axis("off")
    plt.tight_layout()
    plt.show()
```

```
print("\nShowing random sample images from the dataset...\n")
show_samples(train_raw, numeric_classes, real_names, n=9)
```

OUTPUT:

```
================ CLASS MAPPING TABLE ================
Number of classes: 102
Folder     | Flower Name
-------------------------------------------------
1          | pink primrose
10         | globe thistle
100        | blanket flower
101        | trumpet creeper
102        | blackberry lily
11         | snapdragon
12         | colt's foot
13         | king protea
14         | spear thistle
15         | yellow iris
16         | globe-flower
17         | purple coneflower
18         | peruvian lily
19         | balloon flower
2          | hard-leaved pocket orchid
20         | giant white arum lily
21         | fire lily
22         | pincushion flower
23         | fritillary
```

24 | red ginger

25 | grape hyacinth

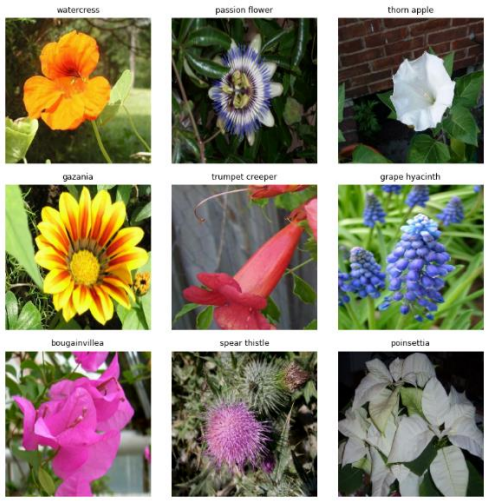
26 | corn poppy

27 | prince of wales feathers

28 | stemless gentian

... (showing 24 of 102 classes)

Showing random sample images from the dataset...



Step 4: Build & Train Model

INPUT:

```
# Model Training

# Loading dataset

train_ds = tf.keras.utils.image_dataset_from_directory(

    os.path.join(base_path, "train"),

    image_size=(img_size, img_size),

    batch_size=batch_size,

    label_mode='int',

    shuffle=True)

valid_ds = tf.keras.utils.image_dataset_from_directory(

    os.path.join(base_path, "valid"),
```

```
    image_size=(img_size, img_size),
    batch_size=batch_size,
    label_mode='int',
    shuffle=True)
test_ds = tf.keras.utils.image_dataset_from_directory(
    os.path.join(base_path, "test"),
    image_size=(img_size, img_size),
    batch_size=batch_size,
    label_mode='int',
    shuffle=False)
# Preprocess function
def preprocess(image, label):
    image = preprocess_input(tf.cast(image, tf.float32))
    label = tf.one_hot(label, num_classes)
    return image, label
train_ds = train_ds.map(preprocess).prefetch(tf.data.AUTOTUNE)
valid_ds = valid_ds.map(preprocess).prefetch(tf.data.AUTOTUNE)
test_ds = test_ds.map(preprocess).prefetch(tf.data.AUTOTUNE)
# Function to set trainable layers
def set_trainable_layers(base_model, unfreeze_percent):
    total_layers = len(base_model.layers)
    num_unfreeze = int(total_layers * unfreeze_percent)
    for i, layer in enumerate(base_model.layers):
        if i < total_layers - num_unfreeze:
            layer.trainable = False
        else:
            layer.trainable = True
```

```
    print(f"Total layers in base model: {total_layers}")
    print(f"Unfreeze percent: {unfreeze_percent*100:.1f}%")
    print(f"No. of layers unfreeze: {num_unfreeze}")
    print(f"Layers freeze: {total_layers - num_unfreeze}")
    if unfreeze_percent != 0:
        print("\nList of unfreeze layers:")
        for i, layer in enumerate(base_model.layers[-num_unfreeze:]):
            print(f"  {i + (total_layers - num_unfreeze)}: {layer.name}")
# Build model
base_model = VGG19(include_top=False, weights='imagenet', input_shape=(img_size, img_size, 3))
# Unfreeze last 30% of layers
set_trainable_layers(base_model, 0.30)
x = GlobalAveragePooling2D()(base_model.output)
x = Dropout(0.3)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.3)(x)
output = Dense(num_classes, activation='softmax')(x)
model = Model(base_model.input, output)
print(f"Total layers in final model: {len(model.layers)}")
def compact_model_summary(model, base_model_name="VGG19", custom_layer_start_idx=-5):
    col1_width = 52
    col2_width = 25
    col3_width = 12
    print(f"{'Layer (type)':<{col1_width}} {'Output Shape':<{col2_width}} {'Param #':>{col3_width}}")
    print("=" * (col1_width + col2_width + col3_width))
    base_output = model.layers[custom_layer_start_idx - 1].output.shape
    total_base_params = sum([l.count_params() for l in model.layers[:custom_layer_start_idx]])
```

```
    print(f"{base_model_name.title()} (Functional)".ljust(col1_width), f"{str(base_output):<{col2_width}}", f"{total_base_params:>{col3_width},}")
    for layer in model.layers[custom_layer_start_idx:]:
        name = layer.name
        layer_type = layer.__class__.__name__
        try:
            output_shape = str(layer.output.shape)
        except:
            output_shape = "N/A"
        params = f"{layer.count_params():,}"
        print(f"{name + ' (' + layer_type + ')':<{col1_width}} {output_shape:<{col2_width}} {params:>{col3_width}}")
    print("=" * (col1_width + col2_width + col3_width))
    total_params = model.count_params()
    trainable_params = np.sum([tf.keras.backend.count_params(w) for w in model.trainable_weights])
    non_trainable_params = np.sum([tf.keras.backend.count_params(w) for w in model.non_trainable_weights])
    print(f"Total params: {total_params:,}")
    print(f"Trainable params: {trainable_params:,}")
    print(f"Non-trainable params: {non_trainable_params:,}")
print("\nPrinting Model Summary:")
compact_model_summary(model, base_model_name="VGG19", custom_layer_start_idx=-5)
# Compile model
loss_fn = tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.1)
ckpt_path = os.path.join(output_dir, "best_model.h5")
callbacks = [
    EarlyStopping(monitor='val_loss', patience=patience, restore_best_weights=True, verbose=0),
    ModelCheckpoint(ckpt_path, save_best_only=True, monitor='val_loss', verbose=0)
]
```

```
model.compile(optimizer=Adam(learning_rate), loss=loss_fn, metrics=['accuracy'])

# Train model

print("\nStarting training...\n")

history = model.fit(

    train_ds,

    validation_data=valid_ds,

    epochs=epochs,

    callbacks=callbacks,

    verbose=0)

print("\nTraining completed.")
```

OUTPUT:

```
Found 6552 files belonging to 102 classes.

Found 818 files belonging to 102 classes.

Found 819 files belonging to 102 classes.

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg19/vgg19_weights_tf_dim_ordering_tf_kernels_notop.h5

[1m80134624/80134624[0m [32m━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━[0m[37m[0m [1m0s[0m 0us/step

Total layers in base model: 22

Unfreeze percent: 30.0%

No. of layers unfreeze: 6

Layers freeze: 16

List of unfreeze layers:

  16: block4_pool

  17: block5_conv1

  18: block5_conv2

  19: block5_conv3
```

20: block5_conv4

21: block5_pool

Total layers in final model: 27

Printing Model Summary:

```
Layer (type)                 Output Shape              Param #
=================================================================
Vgg19 (Functional)          (None, 7, 7, 512)         20,024,384
global_average_pooling2d     (None, 512)               0
(GlobalAveragePooling2D)
dropout (Dropout)            (None, 512)               0
dense (Dense)                (None, 256)               131,328
dropout_1 (Dropout)          (None, 256)               0
dense_1 (Dense)              (None, 102)               26,214
=================================================================
```

Total params: 20,181,926

Trainable params: 9,596,774

Non-trainable params: 10,585,152

Starting training...

Training completed.

Step 5: Evaluate Model

INPUT:

```
# Model Evaluation
print("\nLoading best model for evaluation...")
best_model = tf.keras.models.load_model(ckpt_path)
print("\nEvaluating on Train:")
train_loss, train_acc = best_model.evaluate(train_ds, verbose=0)
print(f"Train Accuracy: {train_acc*100:.2f}%")
print("\nEvaluating on Validation:")
val_loss, val_acc = best_model.evaluate(valid_ds, verbose=0)
print(f"Validation Accuracy: {val_acc*100:.2f}%")
print("\nEvaluating on Test:")
test_loss, test_acc = best_model.evaluate(test_ds, verbose=0)
print(f"Test Accuracy: {test_acc*100:.2f}%")
```

OUTPUT:

```
Loading best model for evaluation...

Evaluating on Train:
Train Accuracy: 100.00%

Evaluating on Validation:
Validation Accuracy: 94.50%

Evaluating on Test:
Test Accuracy: 93.28%
```

Train: 100.00%

Validation: 94.50%

Test: 93.28%

Step 6: Visualize Results

INPUT:

```
# Visualizing Results

# Training curves

def plot_training_curves(history):
    acc = history.history["accuracy"]
    val_acc = history.history["val_accuracy"]
    loss = history.history["loss"]
    val_loss = history.history["val_loss"]
    epochs_range = range(1, len(acc) + 1)
    plt.figure(figsize=(14, 5))
    # Accuracy
    plt.subplot(1, 2, 1)
    plt.plot(epochs_range, acc, label="Train Acc")
    plt.plot(epochs_range, val_acc, label="Val Acc")
    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title("Training vs Validation Accuracy")
    plt.legend()
    # Loss
    plt.subplot(1, 2, 2)
    plt.plot(epochs_range, loss, label="Train Loss")
    plt.plot(epochs_range, val_loss, label="Val Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss")
```

```
    plt.title("Training vs Validation Loss")
    plt.legend()
    plt.tight_layout()
    plt.show()
print("\nPlotting training curves...")
plot_training_curves(history)
# Classification report
y_true = []
y_pred = []
for x_batch, y_batch in test_ds:
    preds = best_model.predict(x_batch, verbose=0)
    y_pred.extend(np.argmax(preds, axis=1))
    y_true.extend(np.argmax(y_batch.numpy(), axis=1))
y_true = np.array(y_true)
y_pred = np.array(y_pred)
print("\nClassification Report:")
print(classification_report(y_true, y_pred, target_names=class_names, zero_division=0))
```

OUTPUT:

Plotting training curves...

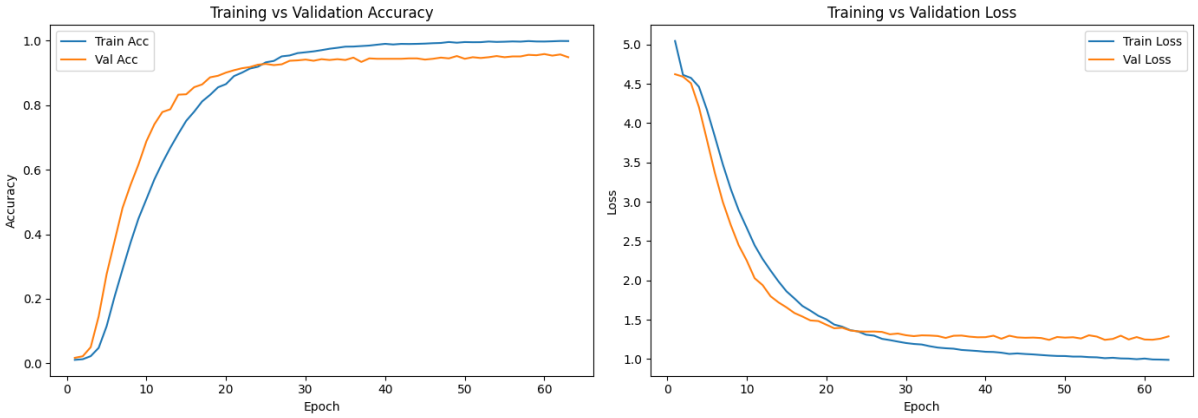
Classification Report:

	precision	recall	f1-score	support
pink primrose	1.00	1.00	1.00	5
globe thistle	1.00	1.00	1.00	3
blanket flower	1.00	1.00	1.00	8
trumpet creeper	1.00	0.75	0.86	4
blackberry lily	1.00	1.00	1.00	6

snapdragon	0.88	0.78	0.82	9
colt's foot	1.00	0.89	0.94	9
king protea	1.00	0.83	0.91	6
spear thistle	0.75	1.00	0.86	3
yellow iris	0.80	1.00	0.89	4
globe-flower	1.00	1.00	1.00	3
purple coneflower	1.00	1.00	1.00	9
peruvian lily	0.83	0.83	0.83	6

... (102 classes total)

accuracy			0.93	819
macro avg	0.93	0.93	0.92	819
weighted avg	0.94	0.93	0.93	819



Step 7: Confusion Matrix

INPUT:

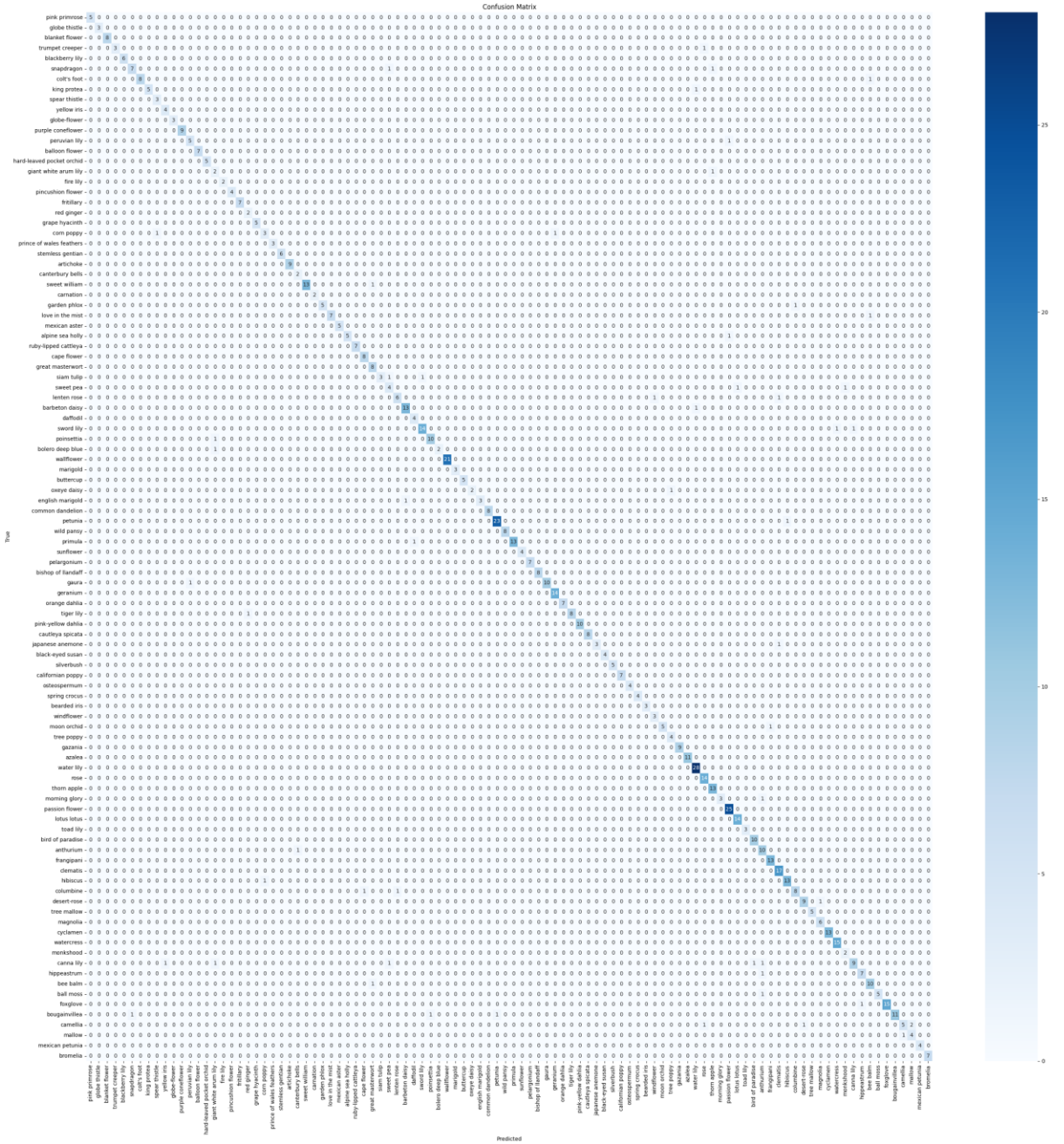
Plotting Confusion Matrix

from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)

```
plt.figure(figsize=(30, 30))
sns.heatmap(cm, cmap="Blues", annot=True, fmt='d',
        xticklabels=class_names, yticklabels=class_names,)
plt.xlabel("Predicted")
plt.ylabel("True")
plt.title("Confusion Matrix")
plt.tight_layout()
plt.show()
```

OUTPUT:


Confusion Matrix
True
Predicted
pink primrose
globe thistle
blanket flower
trumpet creeper
blackberry lily
snapdragon
colt's foot
king protea
spear thistle
yellow iris
globe-flower
purple coneflower
peruvian lily
balloon flower
hard-leaved pocket orchid
giant white arum lily
fire lily
pincushion flower
fritillary
red ginger
grape hyacinth
corn poppy
prince of wales feathers
stemless gentian
artichoke
canterbury bells
sweet william
carnation
garden phlox
love in the mist
mexican aster
alpine sea holly
ruby-lipped cattleya
cape flower
great masterwort
siam tulip
sweet pea
lenten rose
barbeton daisy
daffodil
sword lily
poinsettia
bolero deep blue
wallflower
marigold
buttercup
oxeye daisy
english marigold
common dandelion
petunia
wild pansy
primula
sunflower
pelargonium
bishop of llandaff
gaura
geranium
orange dahlia
tiger lily
pink-yellow dahlia
cautleya spicata
japanese anemone
black-eyed susan
silverbush
californian poppy
osteospermum
spring crocus
bearded iris
windflower
moon orchid
tree poppy
gazania
azalea
water lily
rose
thorn apple
morning glory
passion flower
lotus lotus
toad lily
bird of paradise
anthurium
frangipani
clematis
hibiscus
columbine
desert-rose
tree mallow
magnolia
cyclamen
watercress
monkshood
canna lily
hippeastrum
bee balm
ball moss
foxglove
bougainvillea
camellia
mallow
mexican petunia
bromelia
25
20
15
10
5
0